EventNova — Diagrama de clases UML definitivo

Clases del dominio con los atributos de develop.sql y los métodos de backend/models · actualizado con la suspensión de usuarios

Pais
tabla PAIS
- idPais: int
- nombrePais: String

+ getAll(): Pais[]
+ getById(id): Pais
+ existsByNombre(nombre): boolean
+ create(nombre): Pais
+ update(id, nombre): Pais
+ remove(id): void

se divide en 1 — 0..*

Departamento
tabla DEPARTAMENTO
- idDepartamento: int
- nombreDepartamento: String

+ getAll(): Departamento[]
+ getByPais(idPais): Departamento[]
+ getById(id): Departamento
+ existsByNombre(nombre, idPais): boolean
+ create(nombre, idPais): Departamento
+ update(id, nombre, idPais): Departamento
+ remove(id): void

contiene 1 — 0..*

Ciudad
tabla CIUDAD
- idCiudad: int
- nombreCiudad: String

+ getAll(): Ciudad[]
+ getByDepartamento(idDep): Ciudad[]
+ getById(id): Ciudad
+ existsByNombre(nombre, idDep): boolean
+ create(nombre, idDep): Ciudad
+ update(id, nombre, idDep): Ciudad
+ remove(id): void

«servicio · sin tabla»
Reporte

+ totales(): Totales
+ ingresosPorMes(anio): Serie
+ reservasPorMes(anio): Serie
+ eventosPorMes(anio): Serie
+ ingresosPorAgente(anio): Fila[]
+ ingresosPorCliente(anio): Fila[]
+ reservasPorEventoMes(anio): Fila[]
+ eventosPorAgenteCiudad(): Fila[]
+ cobertura(): Fila[]
+ eventosPorEstado(): Fila[]
+ reservasPorEstado(): Fila[]

Convenciones UML

—▷	Herencia: Cliente, Agente y Administrador son un Usuario
◆—	Composición: la parte no existe sin el todo
———	Asociación con multiplicidad (1, 0..1, 0..*, 1..3)
- - ->	Dependencia «usa» (consultas de reportes)
- / +	Atributo privado / método público · / = derivado
Cursiva	Clase abstracta (no se instancia sola)
Métodos	Los de backend/models/*.js (nombres reales del código)
{ … }	Restricción de negocio del método
	Nuevo: suspensión de usuarios y observaciones de reservas

reside en 0..1 — 0..*

«usa»

1

«abstracta»
Usuario
tabla USUARIO
- numeroId: String {PK}
- correo: String {único}
- nombre: String
- contraseña: String (hash bcrypt)
- fechaNacimiento: Date
- direccion: String
- fechaRegistro: DateTime
- activo: boolean (false = suspendida)
- fechaSuspension: DateTime
- motivoSuspension: String

/ edad: int {derivado}

+ findByEmail(correo): Usuario
+ existsByEmailOrId(correo, id): boolean
+ listar(rol, buscar): Usuario[]
+ obtener(numeroId): Usuario
+ crear(rol, datos): Usuario
+ actualizar(numeroId, rol, datos): Usuario
+ cambiarActivo(numeroId, activo, motivo): Usuario
+ actualizarContrasena(id, hash): void
+ tieneRol(numeroId, rol): boolean {solo cuentas activas}

TipoPlan
tabla TIPO_PLAN
- idPlan: int
- nombrePlan: String
- descripcionPlan: String
- precioPlan: decimal
- limiteEvento: int (9999 = ilimitado)

+ listar(): TipoPlan[]
+ findByNombre(nombre): TipoPlan
+ getById(id): TipoPlan
+ cupoDeAgente(numeroId): Cupo {solo agentes activos}

1 — tiene — 1..3

suscribe 1 — 0..*

Cliente
tabla CLIENTE
- codigoCliente: String
- puntos: int
- visualizarPublicidad: boolean

+ createCliente(datos): Cliente

Administrador
tabla ADMINISTRADOR
- codigoAdmin: String
- salario: decimal
- horario: String

+ esAdministrador(numeroId): boolean
+ contarAdministradores(): int

Telefono
tabla TELEFONOS
- telefono: String

Agente
tabla AGENTE
- nombreEmpresa: String
- descripcionAgente: String
- comision: decimal (%)
- experiencia: int (años)

+ createAgente(datos): Agente
+ estadisticasAgente(numeroId): Resumen

«catálogo»
EstadoEvento
tabla ESTADO
- idEstado: int
- nombreEstado: String

1 — crea — 0..*

1 — se encuentra — 0..*

«usa»

se realiza en

«usa»

Evento
tabla EVENTO
- idEvento: int {PK}
- codigoEvento: String {único, EVT-00001}
- nombreEvento: String
- descripcionEvento: String
- teatro: String
- direccionEvento: String
- fechaInicio: DateTime
- fechaFin: DateTime
- capacidad: int {> 0}
- precioBase: decimal {≥ 0}
- observaciones: String
- imagenEvento: String
- causaCancelacion: String
- fechaCancelacion: DateTime
- fechaRegistro: DateTime

+ getAll(filtros): Evento[]
+ getCatalogo(filtros): Evento[] {agentes activos}
+ getById(id): Evento
+ getEstados(): EstadoEvento[]
+ create(datos, idAgente): Evento
+ update(id, datos): Evento
+ cambiarEstado(id, idEstado, causa): Evento
+ cancelarConReservas(id, idEstado, causa, por): Evento
+ actualizarEstadosAutomaticos(): void
+ tieneReservas(id): boolean
+ remove(id): void

hace

«catálogo»
EstadoReserva
tabla ESTADO_RESERVA
- idEstadoReserva: int
- nombreEstado: String

1 — tiene — 0..*

1 — recibe — 0..*

Reserva
tabla RESERVA
- idReserva: int {PK}
- codigoReserva: String {único, RES-00001}
- fechaHoraReserva: DateTime
- numeroEntradas: int {1..10}
- valorTotal: decimal
- observaciones: String
- causaCancelacion: String
- fechaCancelacion: DateTime
- canceladaPor: String

+ getAll(filtros): Reserva[]
+ getById(id): Reserva
+ getEstados(): EstadoReserva[]
+ create(datos, idCliente, maxEntradas): Reserva
+ updateObservaciones(id, observaciones): Reserva
+ update(id, datos): Reserva
+ cambiarEstado(id, idEstado, causa, por): Reserva
+ cancelarEntradas(id, cantidad, nota, causa, por): Reserva
+ cuposDisponibles(idEvento): int
+ remove(id): void {solo canceladas}

1 — registra — 0..*

CancelacionEntradas
tabla CANCELACION_ENTRADAS
- idCancelacion: int
- entradasCanceladas: int
- entradasAntes: int
- valorCancelado: decimal
- causa: String
- canceladaPor: String
- fechaCancelacion: DateTime

EventNova — Diagrama de paquetes UML: arquitectura MVC

Cómo viaja una petición: navegador (MVC del cliente) → API REST Express (MVC del servidor) → PostgreSQL en Neon · Sprint 3, Tarea 17

